

Design and Analysis of Algorithm

Module IV

Let me know, if any typos and other mistakes in the shared ppt slides

Module 4

Module No. 4	Iterative Improvement	5 Hours
-The Simplex Method -Maximum Matching in Bipartite Graphs -The Maximum-Flow Problem - The Stable Marriage Problem.		

Stable Marriage Problem



Consider a set $Y = \{m_1, m_2, \dots, m_n\}$ of n men and
a set $X = \{w_1, w_2, \dots, w_n\}$ of n women.

Each man has a preference list ordering the women as
potential marriage partners with no ties allowed.

Similarly, each woman has a preference list of the men,
also with no ties.

- Example with 3 men and 3 women

Bob: Lea Ann Sue

Jim:

Tom:

men's preferences

	1st	2nd	3rd
Bob:	Lea	Ann	Sue
Jim:	Lea	Sue	Ann
Tom:	Sue	Lea	Ann

women's preferences

	1st	2nd	3rd
Ann:	Jim	Tom	Bob
Lea:	Tom	Bob	Jim
Sue:	Jim	Tom	Bob

ranking matrix

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Boxed cells composing an unstable matching.

A cell in row **m** and column **w** contains two rankings:

The first is the position (ranking) of **w** in the **men's** preference list;

The second is the position (ranking) of **m** in the **women's** preference list.

Example

men's preferences

	1st	2nd	3rd
Bob:	Lea	Ann	Sue
Jim:	Lea	Sue	Ann
Tom:	Sue	Lea	Ann

women's preferences

	1st	2nd	3rd
Ann:	Jim	Tom	Bob
Lea:	Tom	Bob	Jim
Sue:	Jim	Tom	Bob

Preference
matrix

ranking matrix

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

```
Initialize all men and women to free
while there exist a free man m who still has a woman w to
propose to
{
    w = m's highest ranked such woman to whom he has not yet
    proposed
    if w is free
        (m, w) become engaged
    else some pair (m', w) already exists
        if w prefers m to m'
            (m, w) become engaged
            m' becomes free
        else
            (m', w) remain engaged
}
```

- A marriage matching **M** is called stable if there is no blocking pair for it; otherwise, **M** is called unstable.
- So, this marriage matching is unstable because Bob and Lea can drop their designated mates to join in a union they both prefer.
- The **SMP** is to find a stable marriage matching for men's and women's given preferences.

Example



Free men: Bob, Jim, Tom		Ann	Lea	Sue	Bob proposed to Lea Lea accepted
	Bob	2, 3	<u>1, 2</u>	3, 3	
	Jim	3, 1	1, 3	2, 1	
	Tom	3, 2	2, 1	1, 2	

Free men: Jim, Tom		Ann	Lea	Sue	Jim proposed to Lea Lea rejected
	Bob	2, 3	<u>1, 2</u>	3, 3	
	Jim	3, 1	<u>1, 3</u>	2, 1	
	Tom	3, 2	<u>2, 1</u>	1, 2	

Free men: Jim, Tom		Ann	Lea	Sue	Jim proposed to Sue Sue accepted
	Bob	2, 3	<u>1, 2</u>	3, 3	
	Jim	3, 1	1, 3	<u>2, 1</u>	
	Tom	3, 2	2, 1	1, 2	

		Ann	Lea	Sue	
Free men:	Bob	2, 3	1, 2	3, 3	Tom proposed to Sue Sue rejected
Tom	Jim	3, 1	1, 3	2, 1	
	Tom	3, 2	2, 1	1, 2	

		Ann	Lea	Sue	
Free men:	Bob	2, 3	1, 2	3, 3	Tom proposed to Lea Lea replaced Bob with Tom
Tom	Jim	3, 1	1, 3	2, 1	
	Tom	3, 2	2, 1	1, 2	

		Ann	Lea	Sue	
Free men:	Bob	2, 3	1, 2	3, 3	Bob proposed to Ann Ann accepted
Bob	Jim	3, 1	1, 3	2, 1	
	Tom	3, 2	2, 1	1, 2	

$M = \{(\text{Bob}, \text{Ann}), (\text{Jim}, \text{Sue}), (\text{Tom}, \text{Lea})\}$

Example 2



A	2	4	1	3
B	3	1	4	2
C	2	3	1	4
D	4	1	3	2

Men's Preferences

P	2	1	4	3
Q	4	3	1	2
R	1	4	3	2
S	2	1	4	3

Women's Preferences

Input: A set of n men and a set of n women along with rankings of the women by each man and rankings of the men by each woman with no ties allowed in the rankings

Output: A stable marriage matching

Step 0 Start with all the men and women being free.

Step 1 While there are free men, arbitrarily select one of them and do the following:

Proposal The selected free man m proposes to w , the next woman on his preference list (who is the highest-ranked woman who has not rejected him before).

Response If w is free, she accepts the proposal to be matched with m . If she is not free, she compares m with her current mate. If she prefers m to him, she accepts m 's proposal, making her former mate free; otherwise, she simply rejects m 's proposal, leaving m free.

Step 2 Return the set of n matched pairs.

```
int Mranking[N][N] = {  
    {4, 2, 3, 1},  
    {2, 1, 3, 4},  
    {1, 2, 3, 4},  
    {1, 3, 4, 2},  
};  
int  
ManFree[n]={1,1,1,1};
```

```
int Wranking[N][N] = {  
    {0, 1, 2, 3},  
    {0, 1, 2, 3},  
    {0, 1, 2, 3},  
    {0, 1, 2, 3},  
};  
wPartner[n] ={-1,-1,-1,-1}
```

Algorithm: SMP(Mranking[][], Wranking[][], MenFree[], wPartner[])

```

int cnt = N;
while (cnt > 0)
{
    for (m = 0; m < N; m++)
        if (MenFree[m] == 1) // select first man who is free
            break;
    for (int i = 0; i < N && MenFree[m] == 1; i++)
    {
        int w = Mranking[m][i]; //men's first preference
        if (wPartner[w] == -1) //not paired with any one
        {
            wPartner[w] = m; MenFree[m] = 0; cnt--;
        }
        else
        {
            int CurrPartner = wPartner[w];
            x1=preferenceFun(CurrPartner, w, Wranking);
            x2= preferenceFun(m, w, Wranking);

            if (x1>x2)
            {
                wPartner[w] = m; menFree[m] =0;
                menFree[CurrPartner] =1;
            }
        }
    }
} //for } // while
    
```

```
Int preferenceFun(m, w, Wranging[][])  
{  
    for(i=0;i<N; i++)  
    {  
        if(Wranging[w][i]== m)  
            return i;  
    }  
}
```

- The **time complexity of the stable marriage problem** is $O(N^2)$, where N is the number of men or women.
- However, if all men and women have identical preference lists, a trivial $O(N)$ algorithm suffices

Simplex Method

- Linear programming is a mathematical concept that is used to find the optimal solution of the linear function.
- This method uses simple assumptions for optimizing the given function.
- Linear programming is used in various industries such as shipping industries, manufacturing industries, transportation industries, telecommunications, and others.

Maximize $3x+5y$

subject to: $x+y \leq 4$ and $x+3y \leq 6$

where $x \geq 0$ and $y \geq 0$

Our objective is to find the value of x and y where $3x+5y$ is the maximum.

Here two variables used in the function, so two slack variables (u, v) need to use.

$$1. x+y \leq 4 \rightarrow x+y+u=4$$

$$2. x+3y \leq 6, \rightarrow x+3y+v=6$$

Maximize $3x+5y$

subject to: $x+y \geq 40$ and $x+3y \leq 6$

where $x \geq 0$ and $y \geq 0$

Our objective is to find the value of x and y where $3x+5y$ is the maximum.

Here two variables used in the function, so two slack variables (u, v) need to use.

1. $x+y \geq 40 \rightarrow x+y-u=40$

2. $x+3y \leq 6, \rightarrow x+3y+v=6$

$$\begin{array}{ll}\text{maximize} & 3x + 5y \\ \text{subject to} & x + y \leq 4 \\ & x + 3y \leq 6 \\ & x \geq 0, \quad y \geq 0.\end{array}$$



$$\begin{array}{ll}\text{maximize} & 3x + 5y + 0u + 0v \\ \text{subject to} & x + y + u = 4 \\ & x + 3y + \quad + v = 6 \\ & x, y, u, v \geq 0.\end{array}$$

Initial Table

	C_i	3	4	0	0		
CBi	Slack	x	y	u	v	Sol.	ratio
0	u	1	1	1	0	4	
0	v	1	3	0	1	6	
	Z_j						
Objective row	$Z_j - C_i$						

$C_i \rightarrow$ is the coefficients of variables

$CB_i \rightarrow$ is the coefficients of slack variables

Solution is the same as from the condition

Calculate Zj



	Ci	3	5	0	0		
CBi	Slack	x	y	u	v	Sol.	ratio
0	u	1	1	1	0	4	
0	v	1	3	0	1	6	
	Zj						
Objective row	Zj-Ci						

$$Z_j = \sum_{i=1}^n (CB_i * A_{ij})$$

$$Z_j = 0 * 1 + 0 * 1 = 0$$

Find $Z_j - C_i$ & Check for Optimality



	C_i	3	5	0	0		
CB_i	Slack	x	y	u	v	Sol.	ratio
0	u	1	1	1	0	4	
0	v	1	3	0	1	6	
	Z_j	0	0	0	0	0	
Objective row	$Z_j - C_i$						

$$Z_j - C_i = \quad 0 - 3 = -3 \quad 0 - 5 = -5, \quad 0 - 0 = 0, \quad 0 - 0 = 0$$

**Optimality condition is for max is, all $(Z_j - C_i) \geq 0$
So, optimality not reached, we have to iterate it.**

Identify Pivot row and Pivot column



	Ci	3	5	0	0		
CBi	Slack	x	y	u	v	Sol.	ratio
0	u	1	1	1	0	4	$4/1=4$
0	v	1	3	0	1	6	$6/3=2$
	Zj	0	0	0	0	0	
Objective row	Zj-Ci	-3	-5	0	0	0	

Find the pivot row and pivot column and pivot element

Pivot column = min from objective row ($Z_j - C_i$)

Ratio column = solution / Pivot column element

Pivot row = min from ratio column & Pivot element

Identify the entry variable and exit variable

Entry variable=**Pivot Column variable (i.e Y)**

Exit variable =**Pivot row variable (i.e. V)**

Pivot row new values=old value/Pivot element

Normal row new values=**old value-**

((Corresponding Pivot row element*corresponding column element)/Pivot element)

2nd Table

	Ci	3	5	$4 - ((6 * 1) / 3) = 2$			
CBi	Slack	x	y	u	v	Sol.	ratio
0	u	2/3	0	1	-	2	
5	y				1/3		
	Zj						
Objective row	Zj-Ci						

$$A_{ij} = 1 - ((1 * 1) / 3) = 2/3$$

$$A_{ij} = 1 - ((3 * 1) / 3) = 0$$

Normal row new values=old value-((Corresponding Pivot row element*corresponding column element)/Pivot element)

Pivot row new values=old value /Pivot element

Table: Filling Cells

	Ci	3	5	$4 - ((6 * 1) / 3) = 2$			
CBi	Slack	x	y	u	v	Sol.	ratio
0	u	2/3	0	1	-1/3	2	
5	y	1	3	0	1	6	
	Zj						
Objective row	Zj-Ci						

 A_{ij} for Pivot row $\rightarrow 1/3$
 A_{ij} for Pivot row $\rightarrow 3/3=1$
 A_{ij} for Pivot row $\rightarrow 0/3=0$
 A_{ij} for Pivot row $\rightarrow$
 $1/3=1/3$

Pivot row new values = old value / Pivot element

Pivot Element Identification

Ratio
column=solution/Pivot
column element

	Ci	3	5	0	0	Ratio column=solution/Pivot column element	
CBi	Slack	x	y	u	v	Sol.	ratio
0	u	2/3	0	1	-1/3	2	
5	y	1/3	1	0	1/3	6/3=2	
	Zj						
Objective row	Zj-Ci						

$$Z_j = \sum_{i=1}^n (C_{Bi} * A_{ij})$$

Calculate Zj-Ci

Select Pivot column= min of
~~Objective row~~
 column
 Select Pivot row= min of ratio
 column

As all elements of objective row is not ≥ 0 , so these values are not optimal values of x and y.

Identify the entry variable and exit variable

Entry variable=**Pivot Column variable (i.e x)**

Exit variable =**Pivot row variable (i.e. u)**

Pivot row new values=old value/Pivot element

Normal row new values=**old value-**

((Corresponding Pivot row element*corresponding column element)/Pivot element)

3rd Table

	Ci	3	5	0	0	Pivot Element=2/3	
CBi	Slack	x	y	u	v	Sol.	ratio
3	x						
5	y						
	Zj						
Objective row	Zj-Ci						

Pivot row new values=old value/Pivot element

Normal row new values=old value-((Corresponding Pivot row element*corresponding column element)/Pivot element)

$$Z_j = \sum_{i=1}^n (C_{Bi} * A_{ij})$$

Calculate Zj-Ci

As all elements of objective row is ≥ 0 , so these values are optimal values of x and

y.

The Maximum-Flow Problem

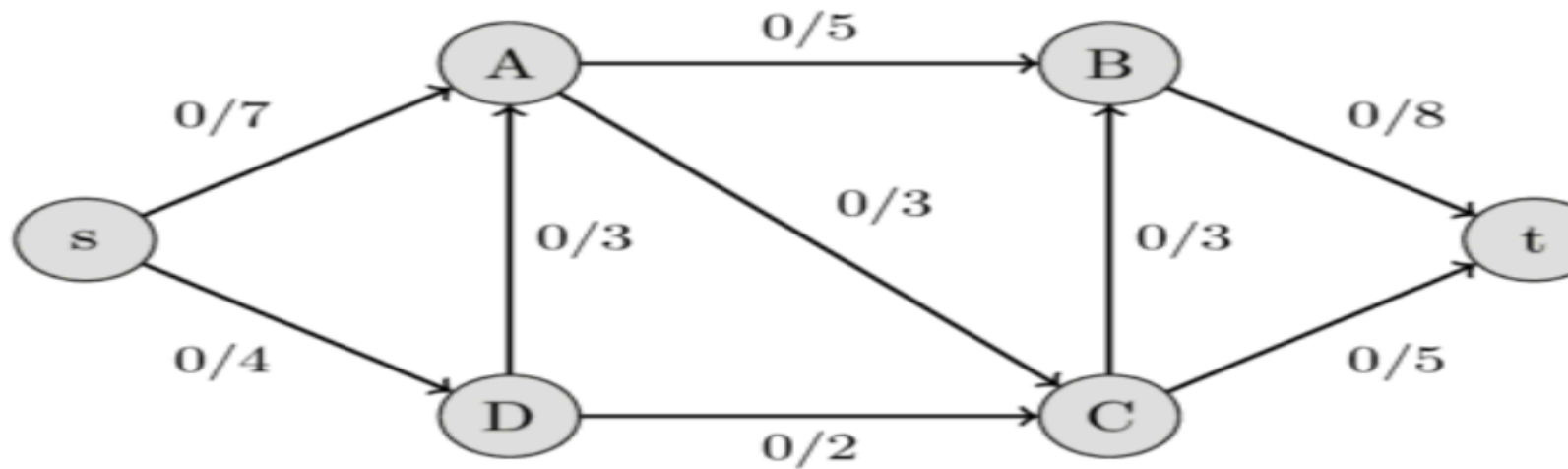
- It is a typical application of graphs is using them to represent networks of transportation.
- In order capture the limitations of the network it is useful to annotate the edges in the graph with capacities that model how much resource can be carried by that connection.
- **The objective is to maximize the flow from source node S to sink node T.**
- **Ford-Fulkerson algorithm**, is used to solve the max flow, which is based on the idea of augmenting paths.

- An augmenting path is a simple path from the source to the sink in the corresponding residual network.
- **Residual network:** it shows the remaining capacity of each edge, which can be used to send additional flow.
- To find the maximum flow in a network, we use the Ford-Fulkerson algorithm, which repeatedly finds an augmenting path in the residual network and increases the flow along that path.

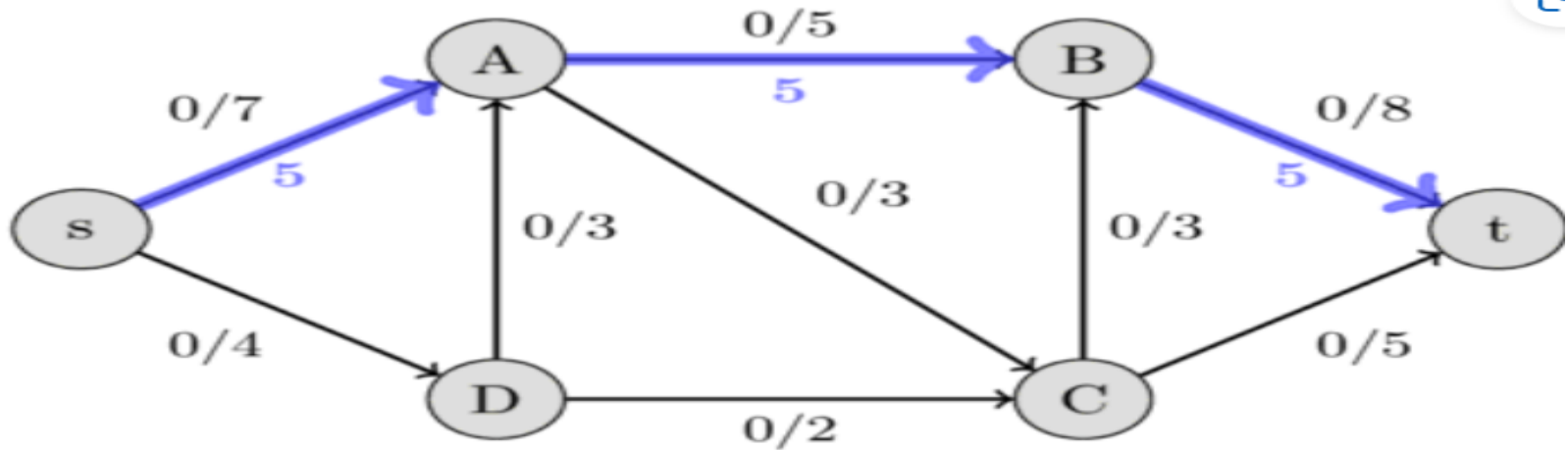
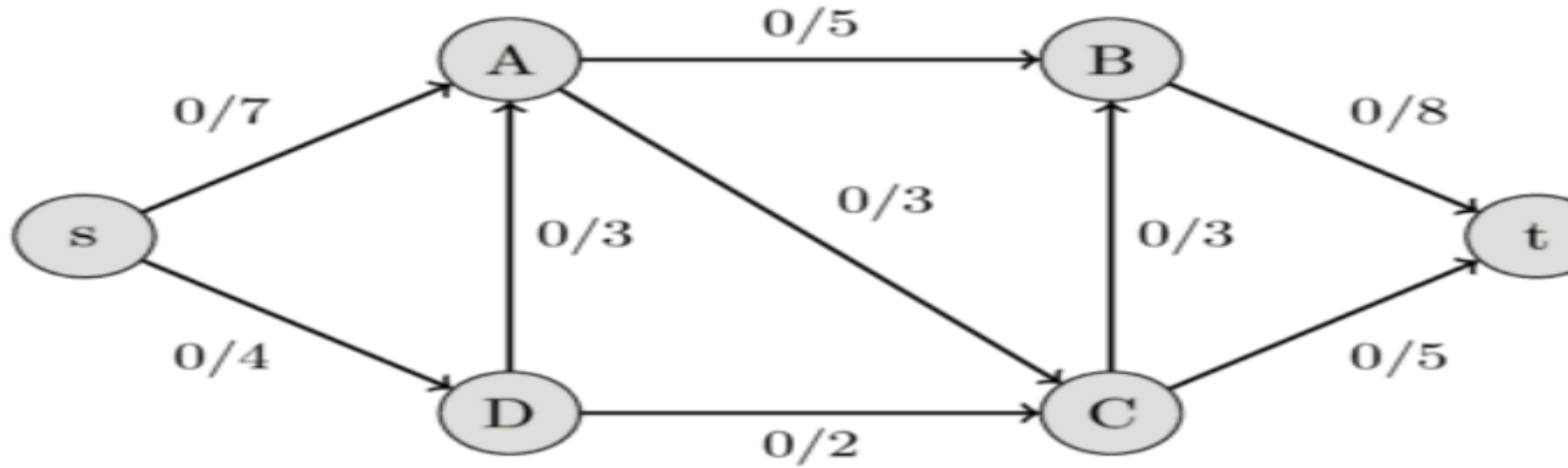
- **Residual Capacity:** It is the capacity of the edge after subtracting the flow from the maximum capacity.
- **Residual Graph:** A graph with the same vertices and same edges, but we use the residual capacities as capacities.

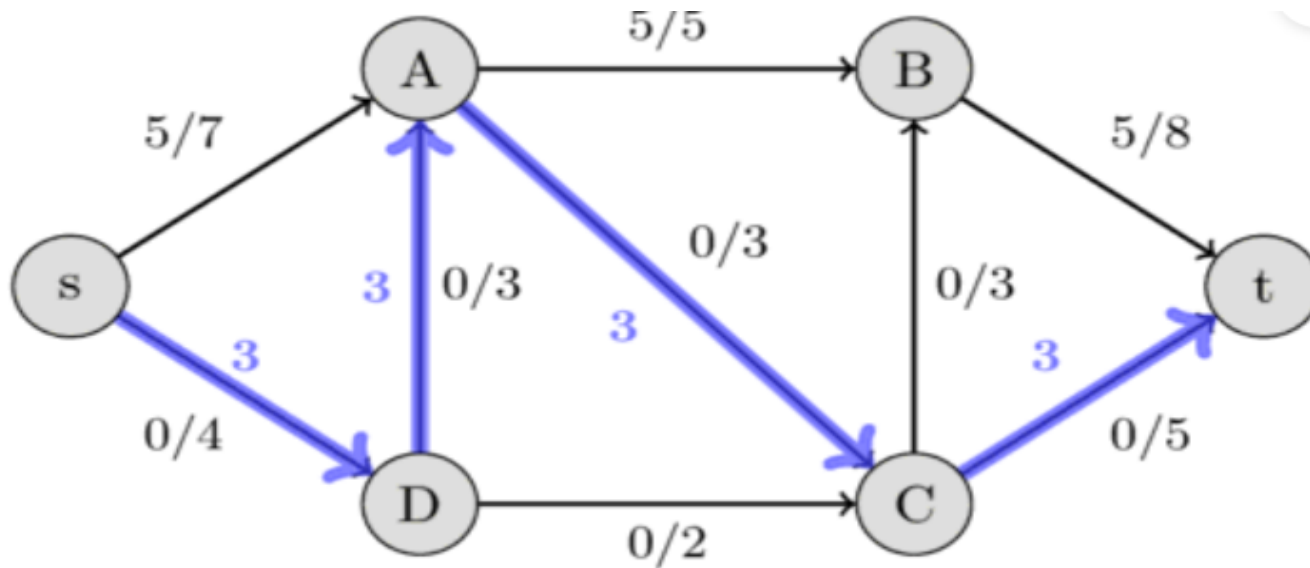
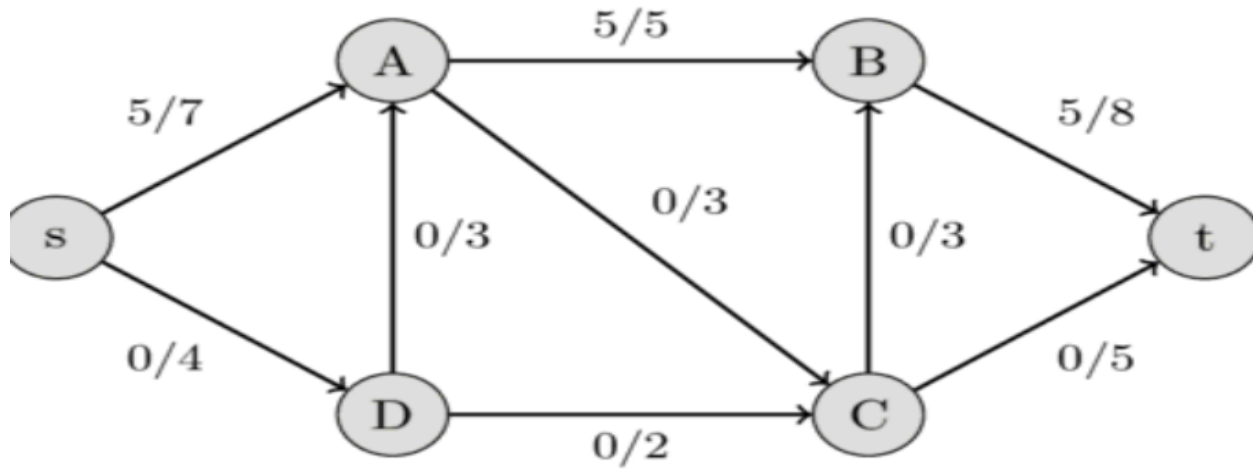
- $\text{Flow} \leq \text{capacity}$
- $\text{Incoming} == \text{Outgoing}$ [except source and sink]
- Non full forward edge
- Non zero backward edge

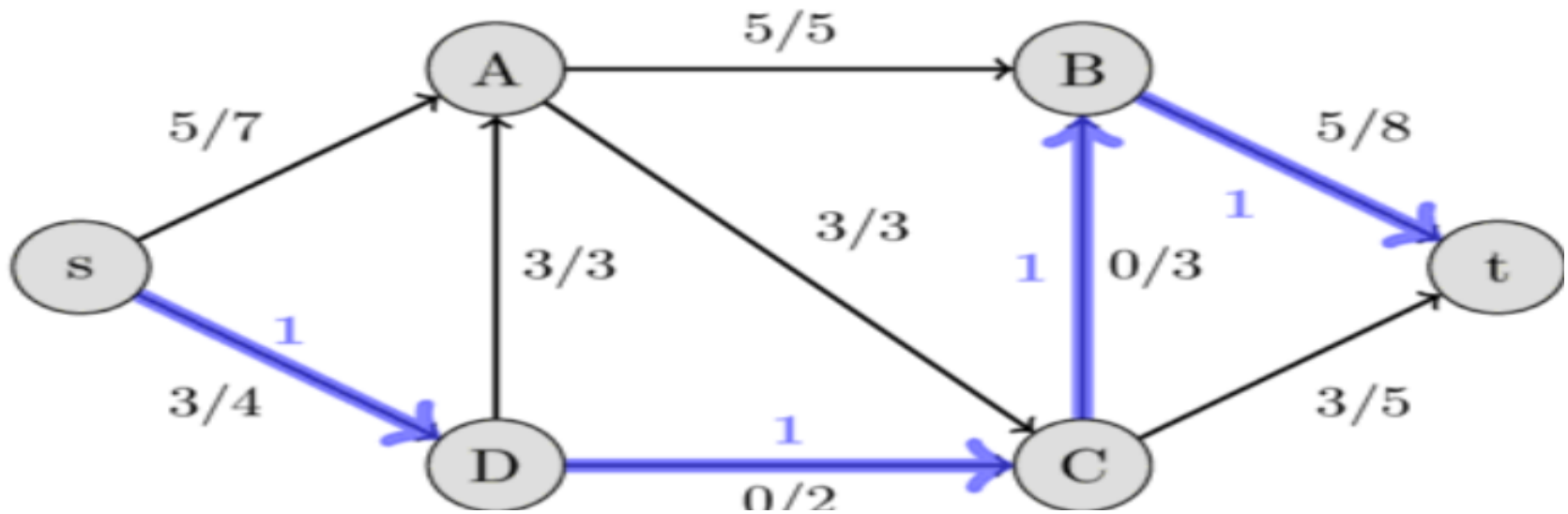
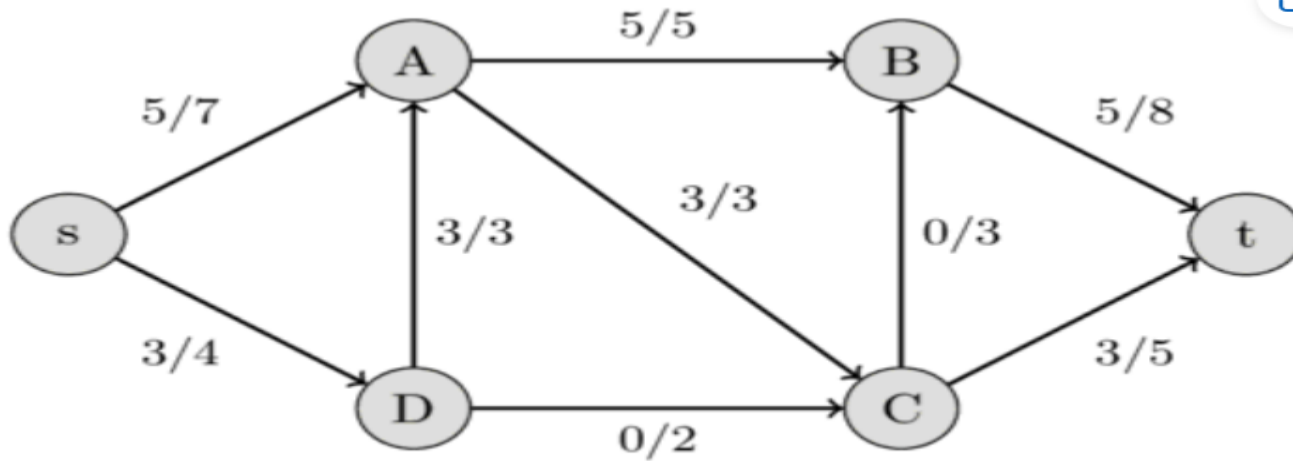
Example

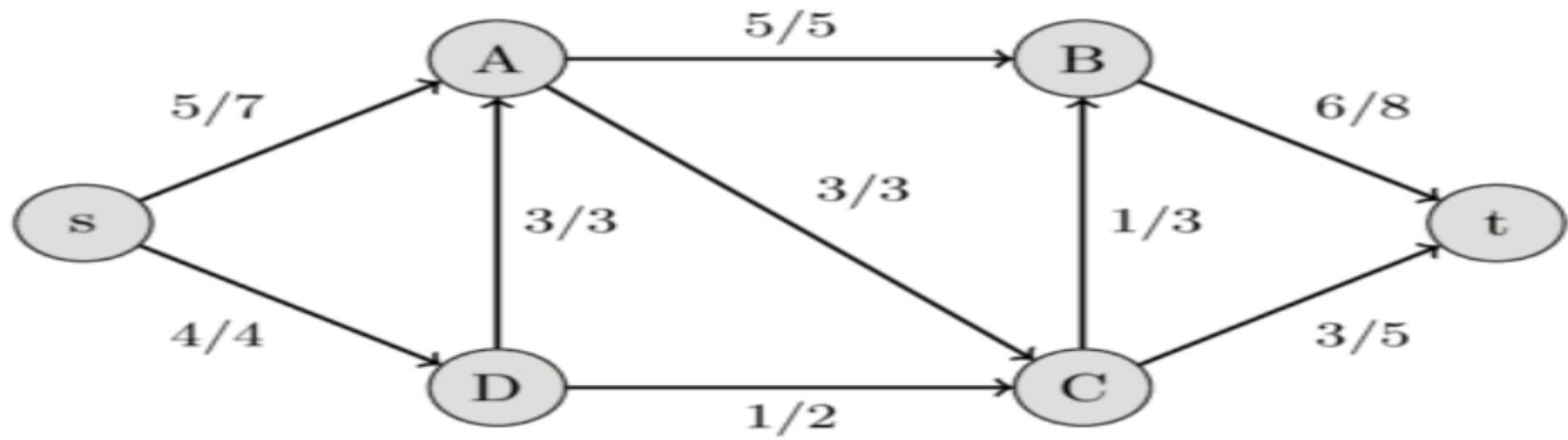


Example









E number of edge
 $f(e)$ flow of edge
 $C(e)$ capacity of edge

1) Initialize : $\text{max_flow} = 0$
 $f(e) = 0$ for every edge 'e' in E

2) Repeat search for an s-t path P while it exists
a) Find if there is a path from s to t using BF or DFS. A path exists if $f(e) < C(e)$ for every edge e on the path.

b) If no path found, return max_flow .

c) Else find minimum edge value for path P

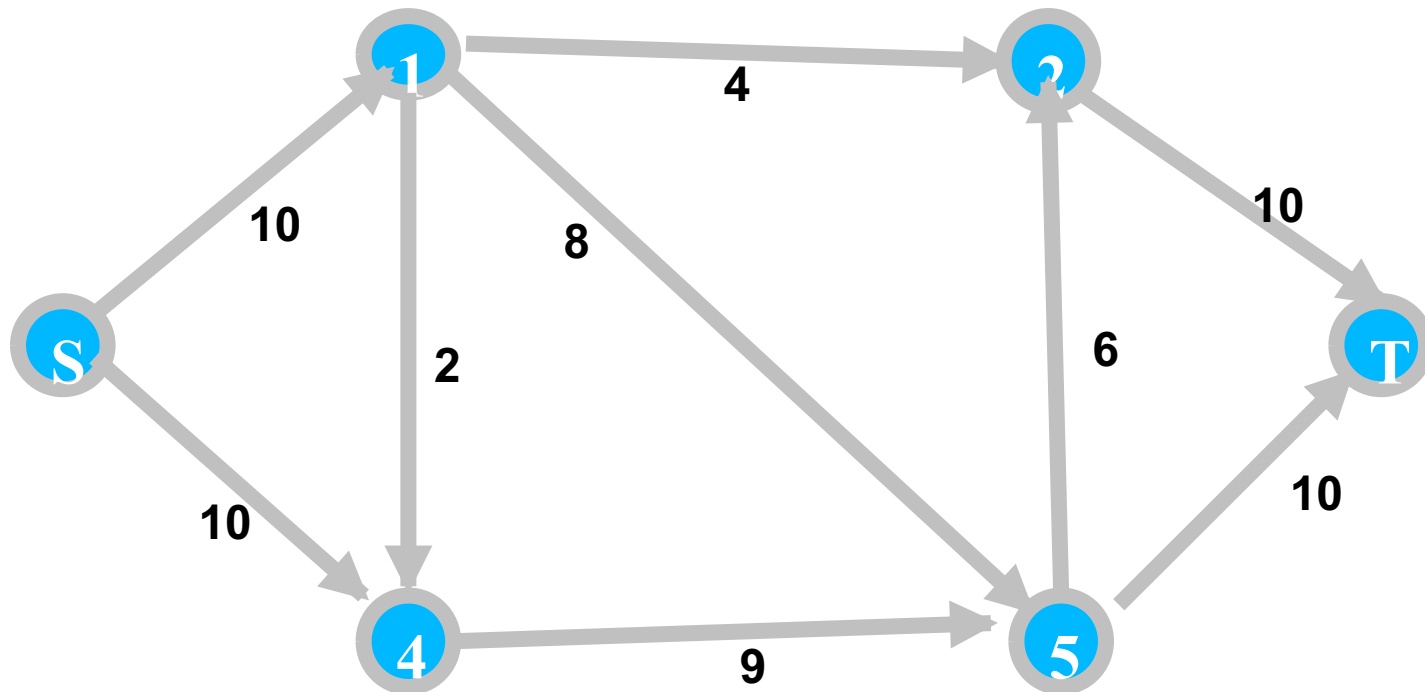
(i) $\text{flow} = \min(C(e) - f(e))$ for path P]
 $\text{max_flow} += \text{flow}$

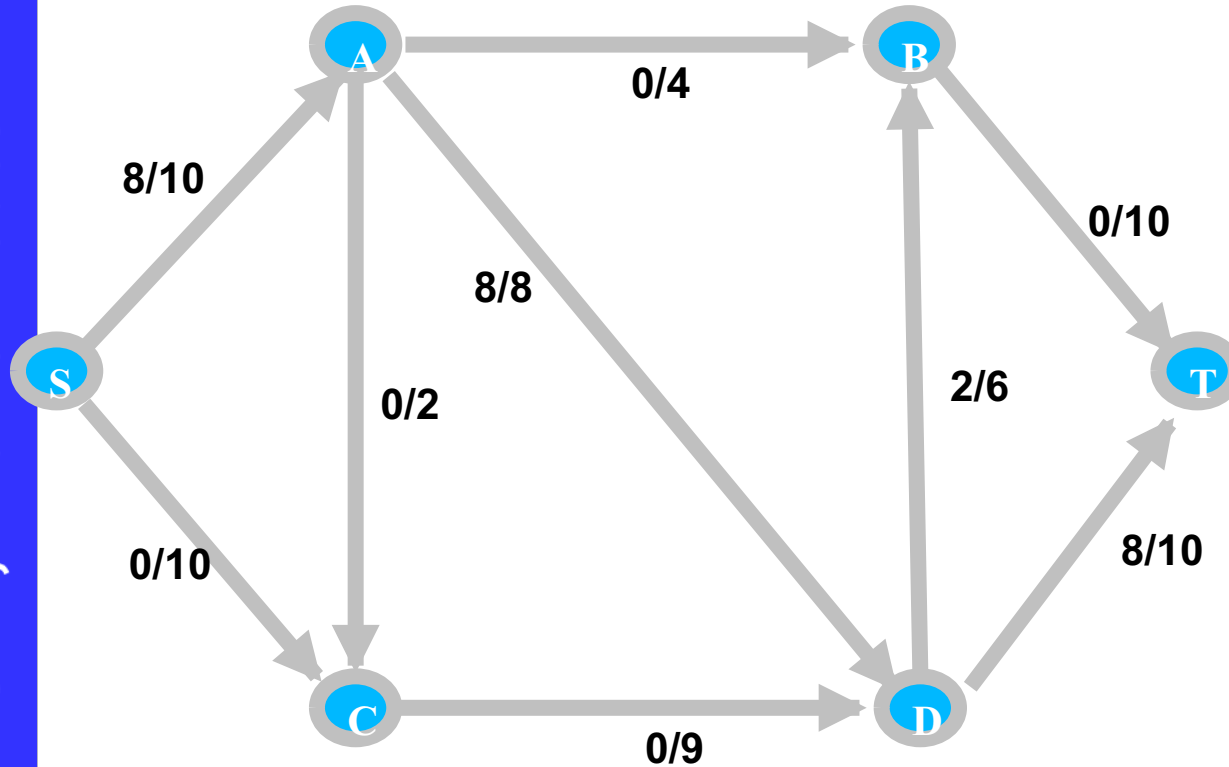
(ii) For all edge e of path increment flow
 $f(e) += \text{flow}$

3) Return max_flow

Example

- Any non-zero backward flow can be considered
- Any non-full forward flow can be considered
- All node's sum of inflow should be equal to sum of outflow



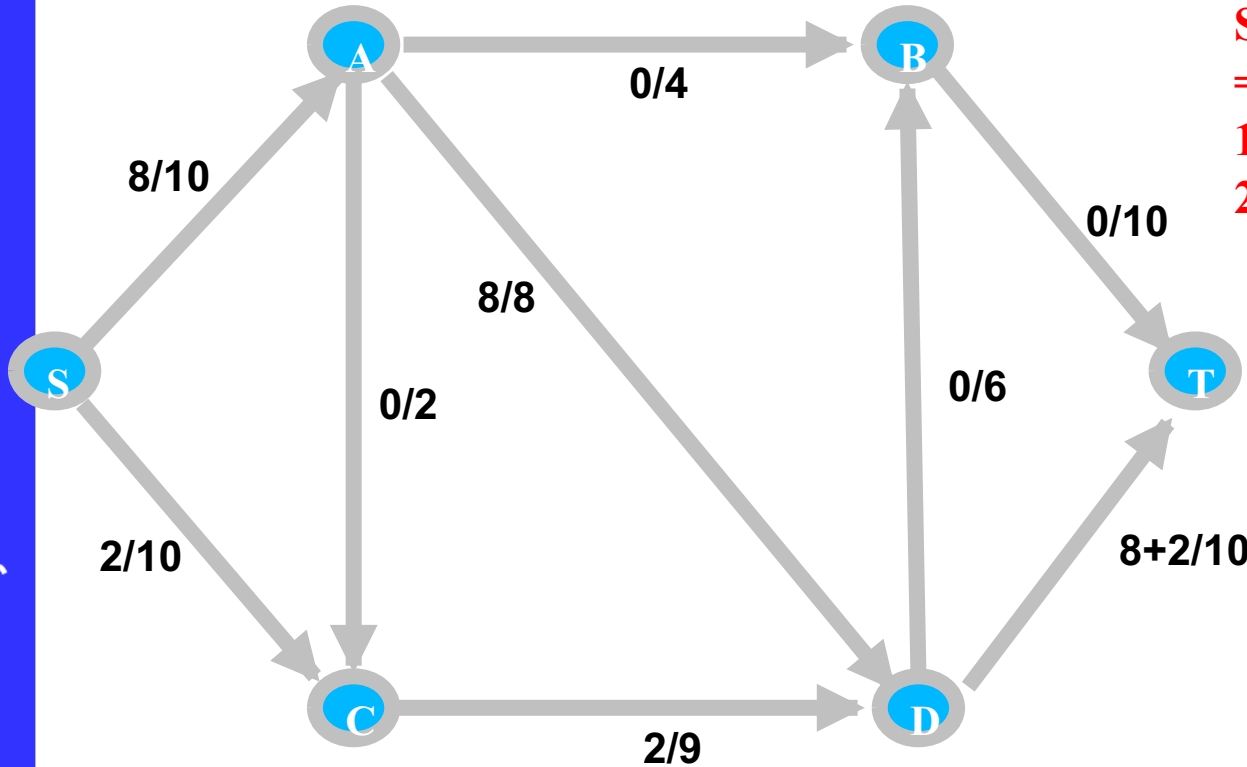


SET OF FLOWS

=====

1. $S \rightarrow A \rightarrow D \rightarrow T = 8$

STEP 2

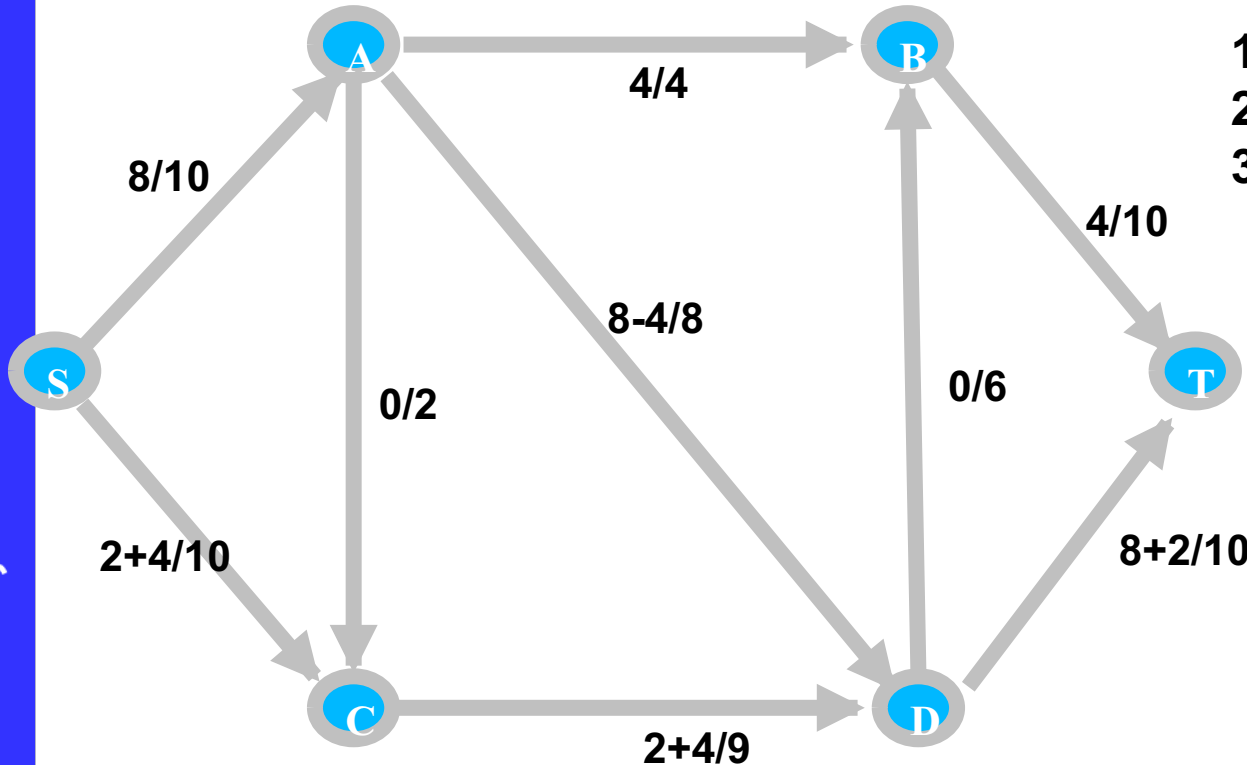


SET OF FLOWS

1. $S \rightarrow A \rightarrow D \rightarrow T = 8$

2. $S \rightarrow C \rightarrow D \rightarrow T = 2$

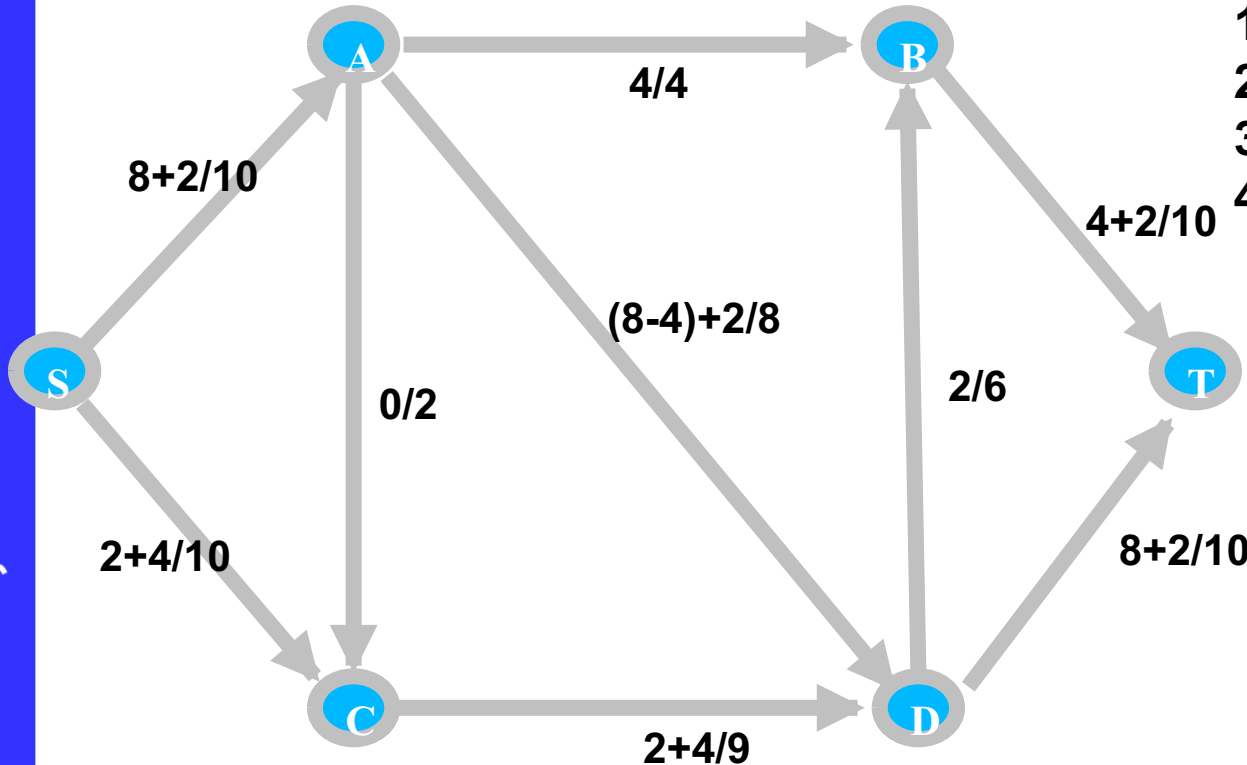
STEP 3



1. $S \rightarrow A \rightarrow D \rightarrow T = 8$
2. $S \rightarrow C \rightarrow D \rightarrow T = 2$
3. $S \rightarrow C \rightarrow D \rightarrow A \rightarrow B \rightarrow T = 4$

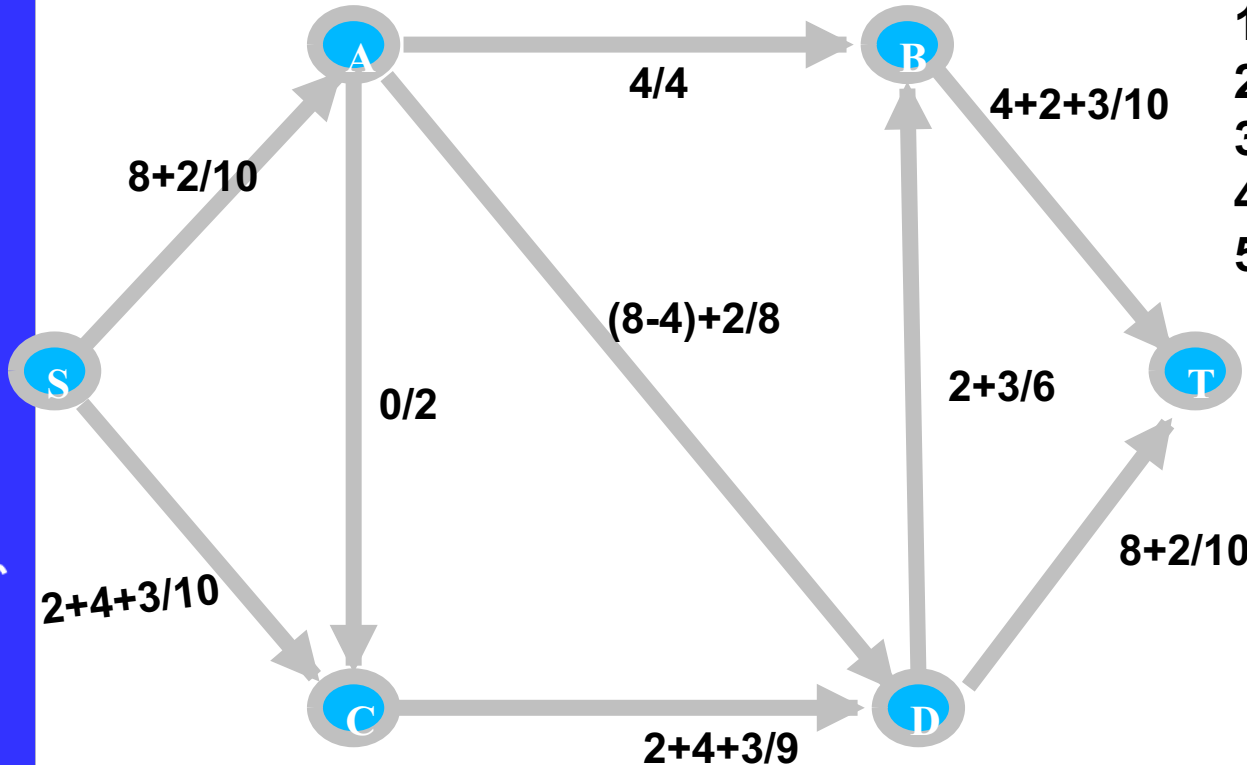
- Any non-zero backward flow can be considered
- Any non-full forward flow can be considered
- All node's sum of inflow should be equal to sum of outflow

STEP 4



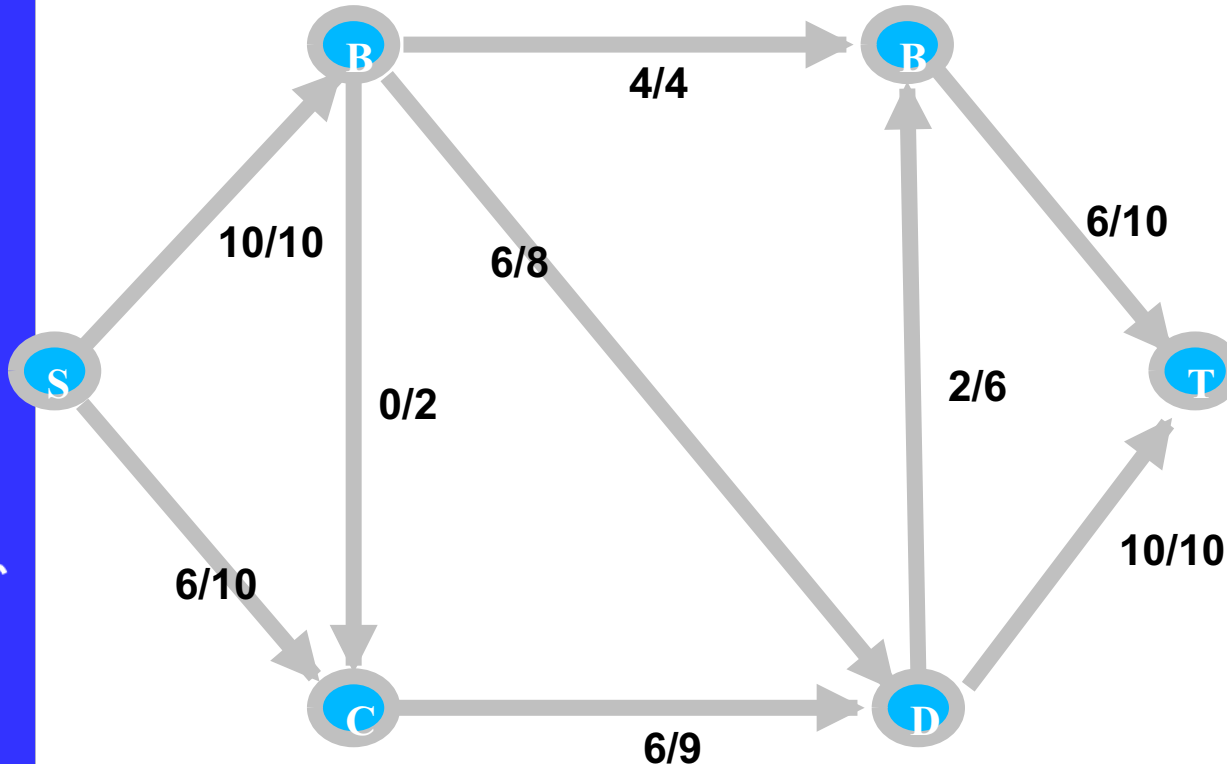
1. $S \rightarrow A \rightarrow D \rightarrow T = 8$
2. $S \rightarrow C \rightarrow D \rightarrow T = 2$
3. $S \rightarrow C \rightarrow D \rightarrow A \rightarrow B \rightarrow T = 4$
4. $S \rightarrow A \rightarrow D \rightarrow B \rightarrow T = 2$

- Any non-zero backward flow can be considered
- **Any non-full forward flow can be considered**
- All node's sum of inflow should be equal to sum of outflow



1. $S \rightarrow A \rightarrow D \rightarrow T = 8$
2. $S \rightarrow C \rightarrow D \rightarrow T = 2$
3. $S \rightarrow C \rightarrow D \rightarrow A \rightarrow B \rightarrow T = 4$
4. $S \rightarrow A \rightarrow D \rightarrow B \rightarrow T = 2$
5. $S \rightarrow C \rightarrow D \rightarrow B \rightarrow T = 3$

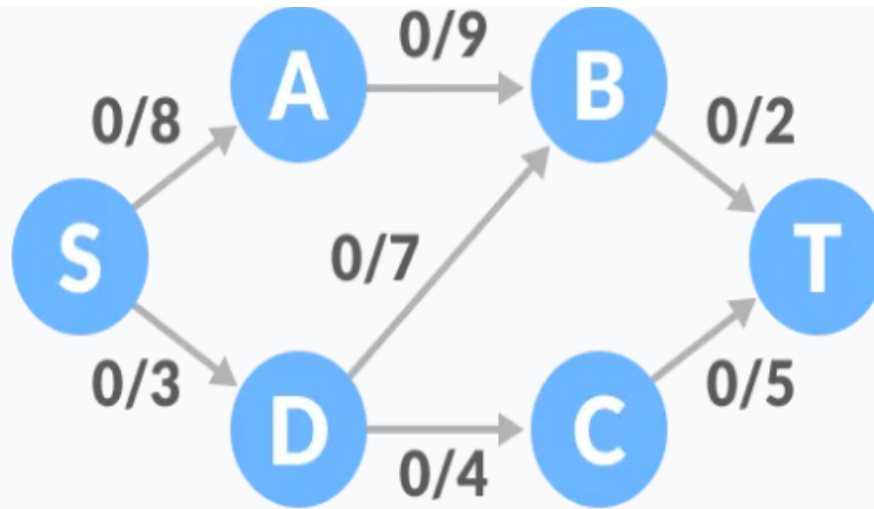
- Any non-zero backward flow can be considered
- Any non-full forward flow can be considered
- All node's sum of inflow should be equal to sum of outflow



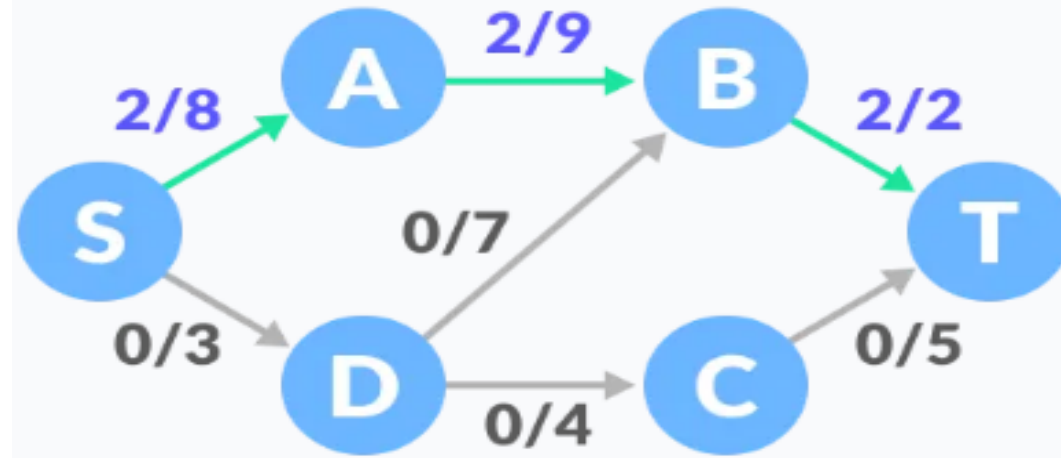
1. $S \rightarrow A \rightarrow D \rightarrow T = 8$
2. $S \rightarrow C \rightarrow D \rightarrow T = 2$
3. $S \rightarrow C \rightarrow D \rightarrow A \rightarrow B \rightarrow T = 4$
4. $S \rightarrow A \rightarrow D \rightarrow B \rightarrow T = 2$
5. $S \rightarrow C \rightarrow D \rightarrow B \rightarrow T = 3$

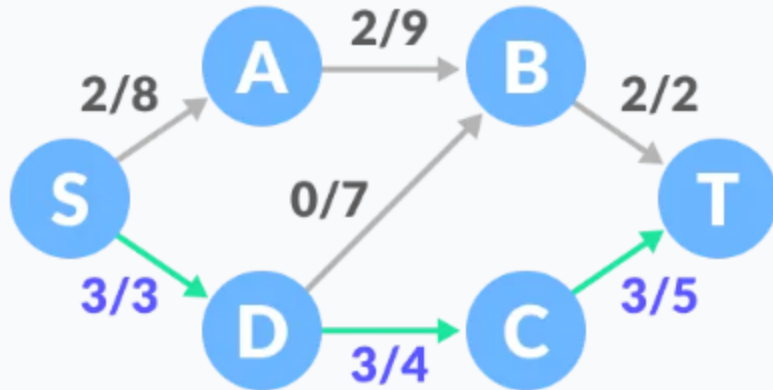
- Any non-zero backward flow can be considered
- Any non-full forward flow can be considered
- All node's sum of inflow should be equal to sum of outflow

Example 3

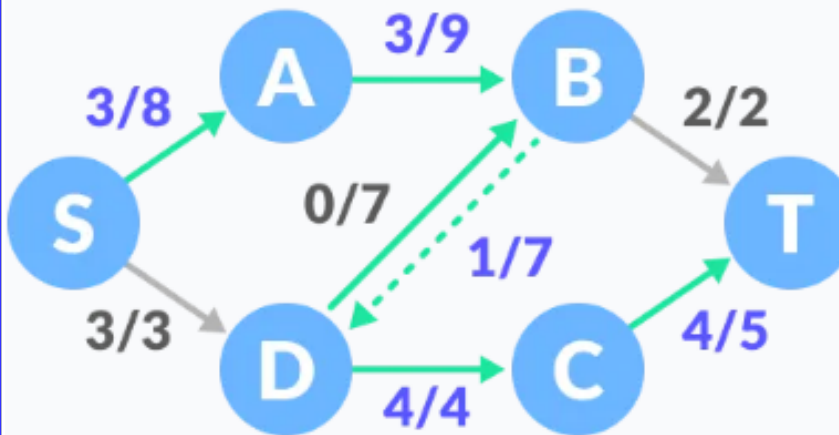


The minimum capacity among the three edges is 2 (B-T). Based on this, update the flow/capacity for each path.





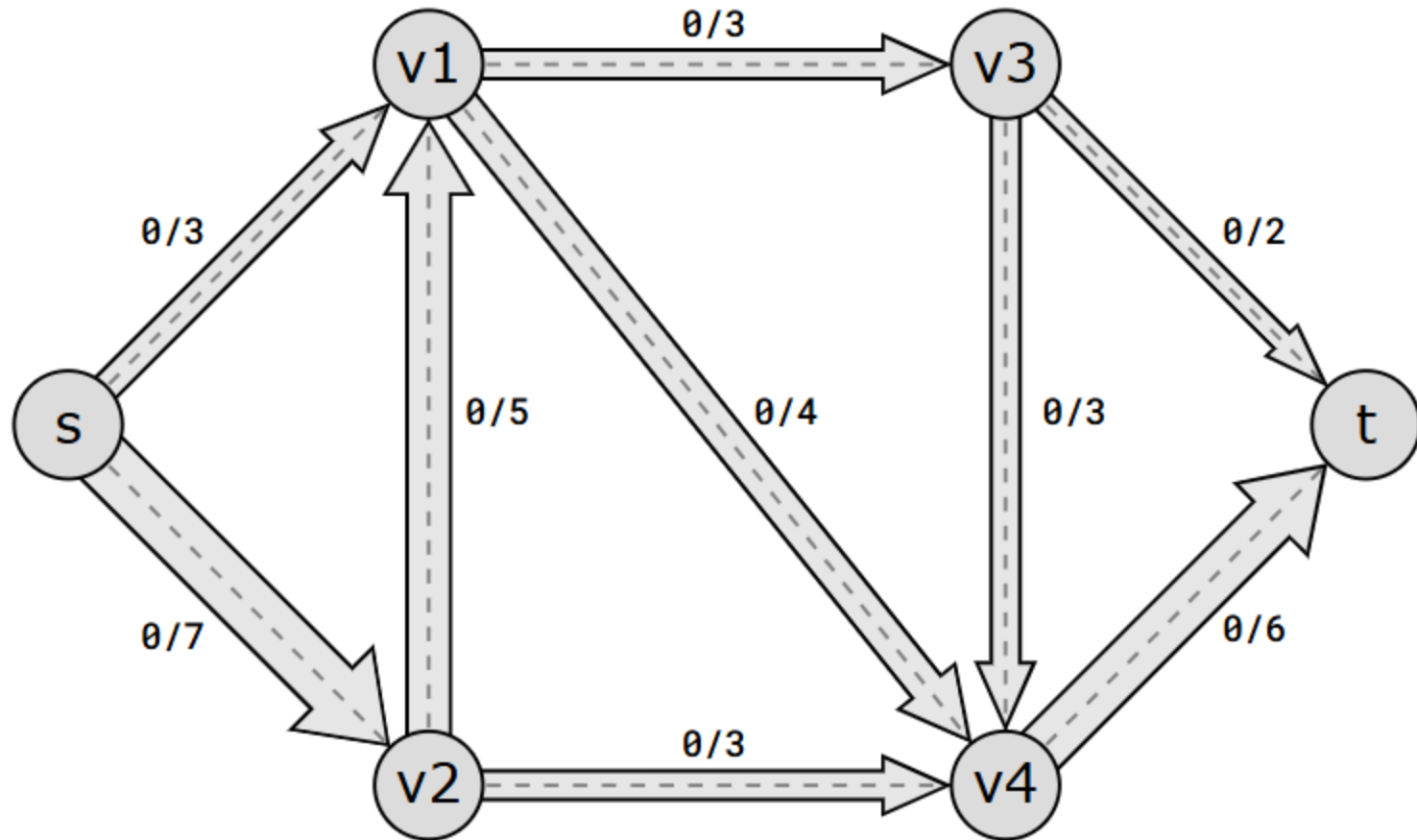
- Select another path S-D-C-T. The minimum capacity among these edges is 3 (S-D)



- Now, let us consider the reverse-path B-D as well. Selecting path S-A-B-D-C-T. The minimum residual capacity among the edges is 1 (D-C).

- Adding all the flows = $2 + 3 + 1 = 6$, which is the maximum possible flow on the flow network.

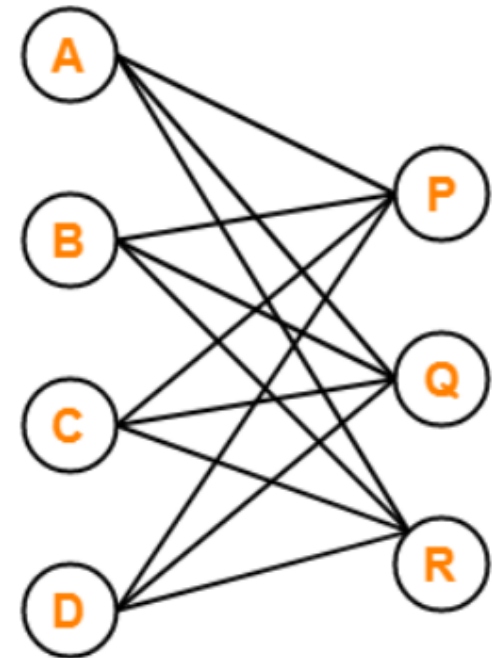
Example 4

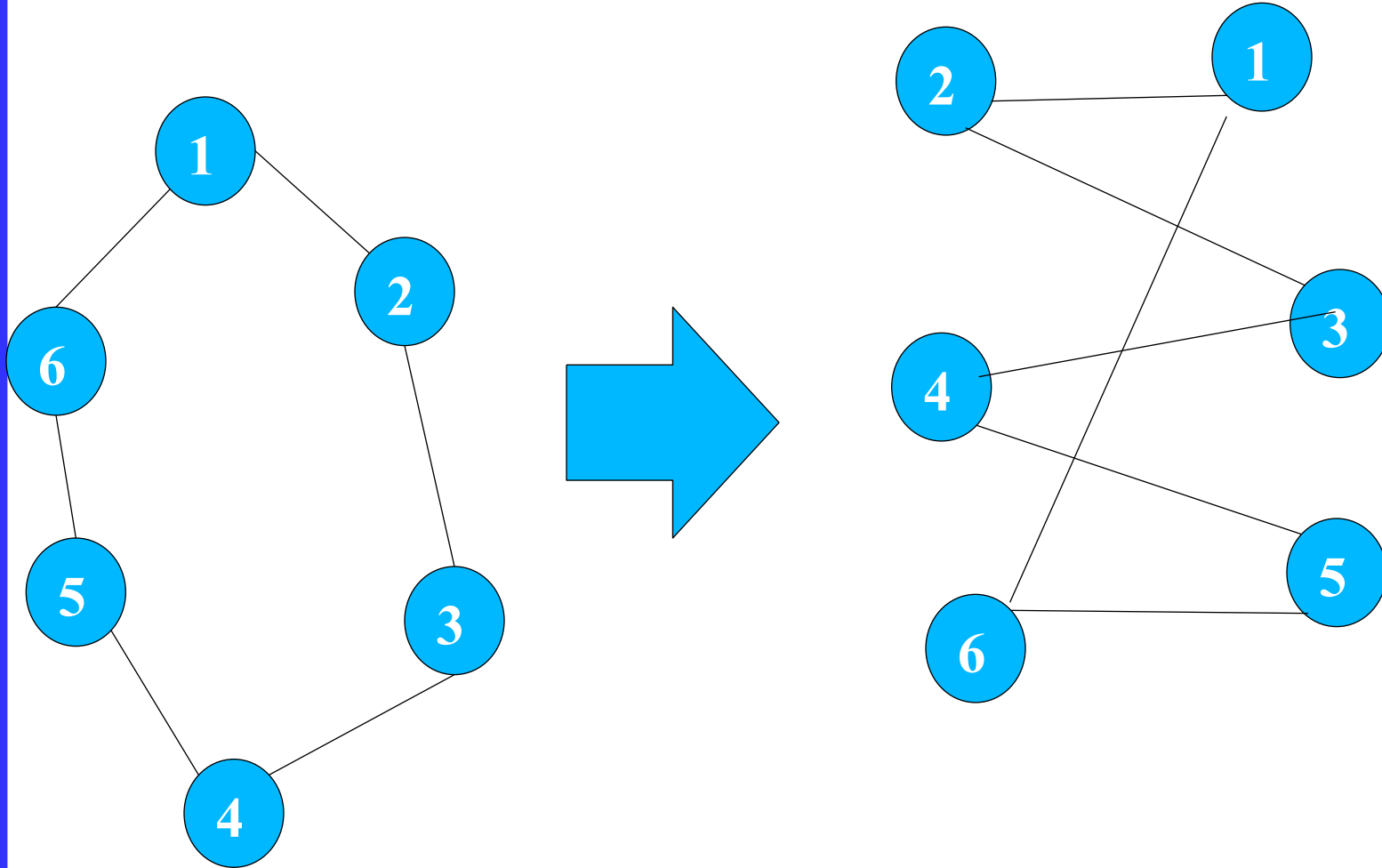




Maximum Matching in Bipartite Graphs

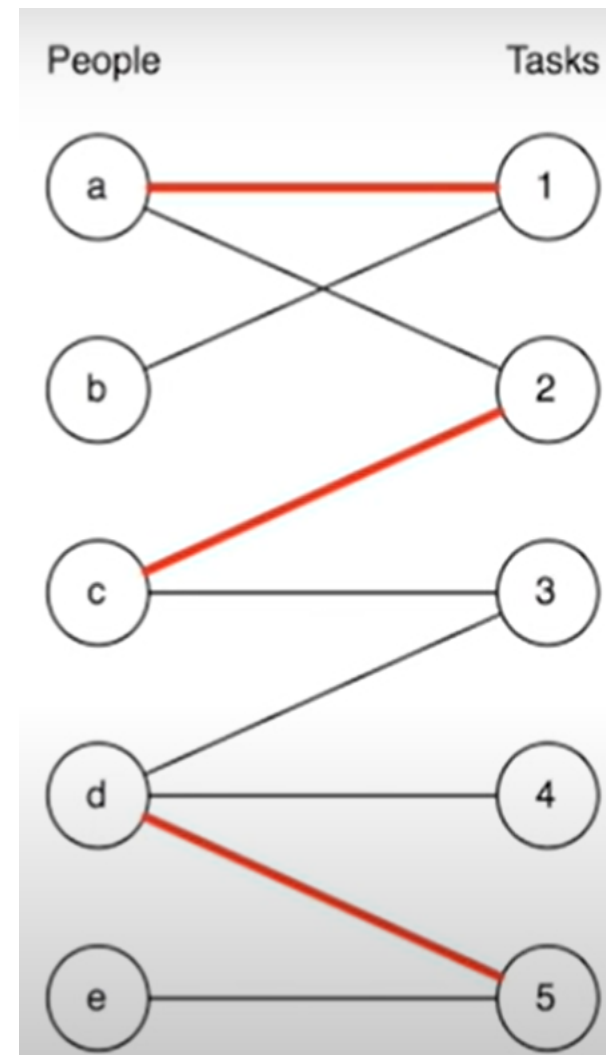
- A bipartite graph is a graph where the vertices can be divided into two disjoint sets such that all edges connect a vertex in one set to a vertex in another set. There are no edges between vertices in





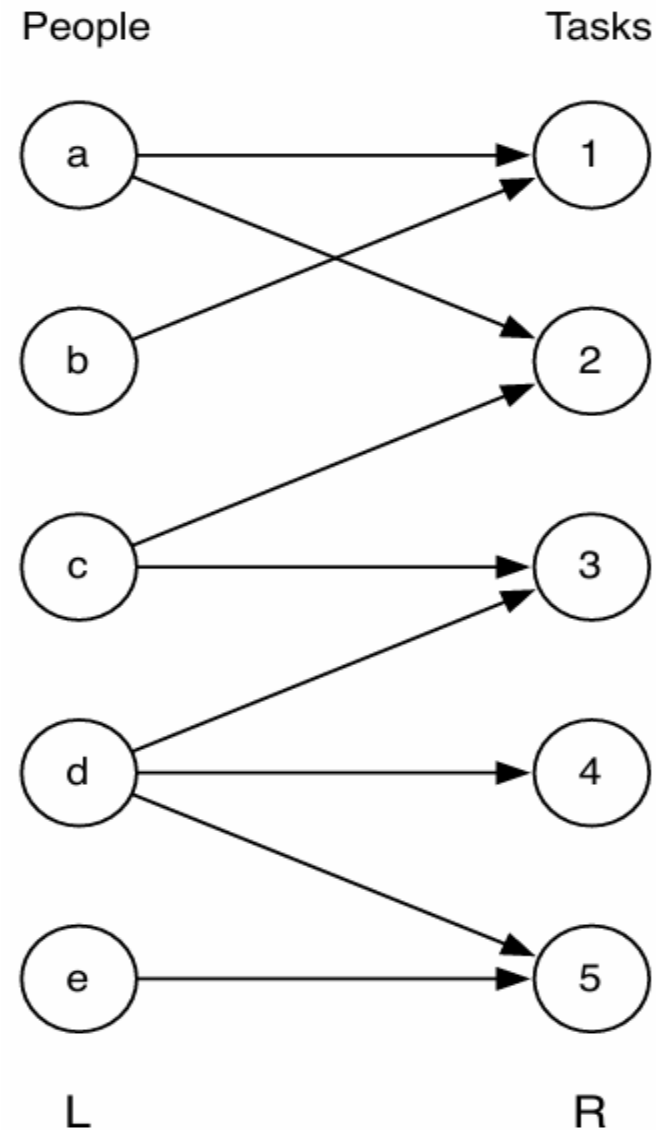
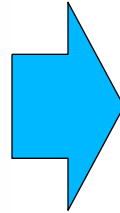
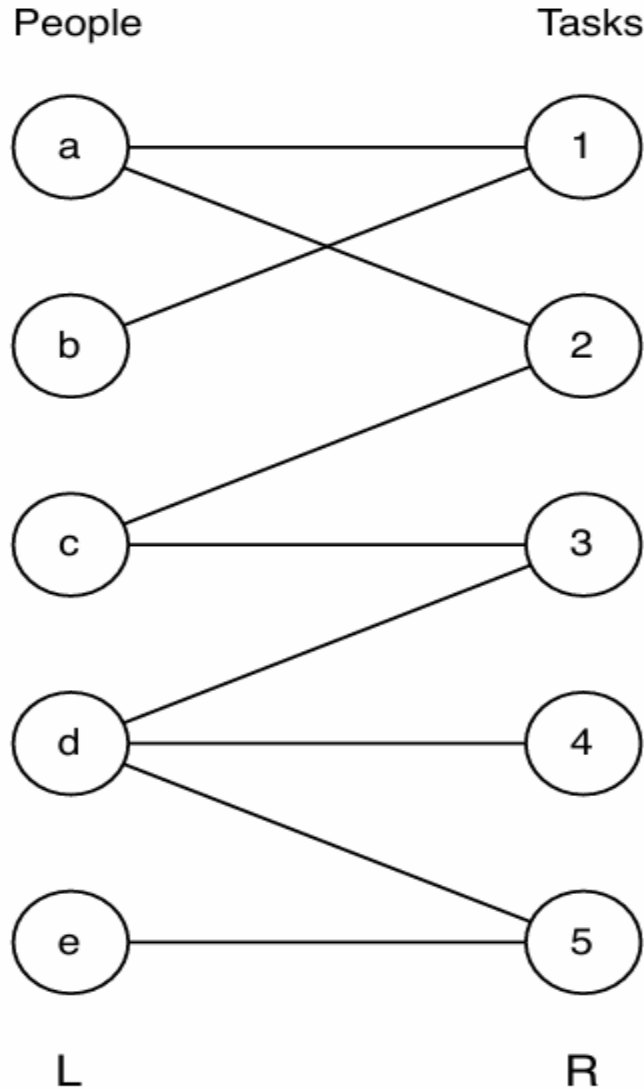
- A maximum matching is a matching of maximum size (maximum number of edges). In a maximum matching, if any edge is added to it, it is no longer a matching.

- A **matching** gives an assignment of people to tasks.
- Want to get as many tasks done as possible.
- So, want a **maximum matching**: one that contains as many edges as possible.

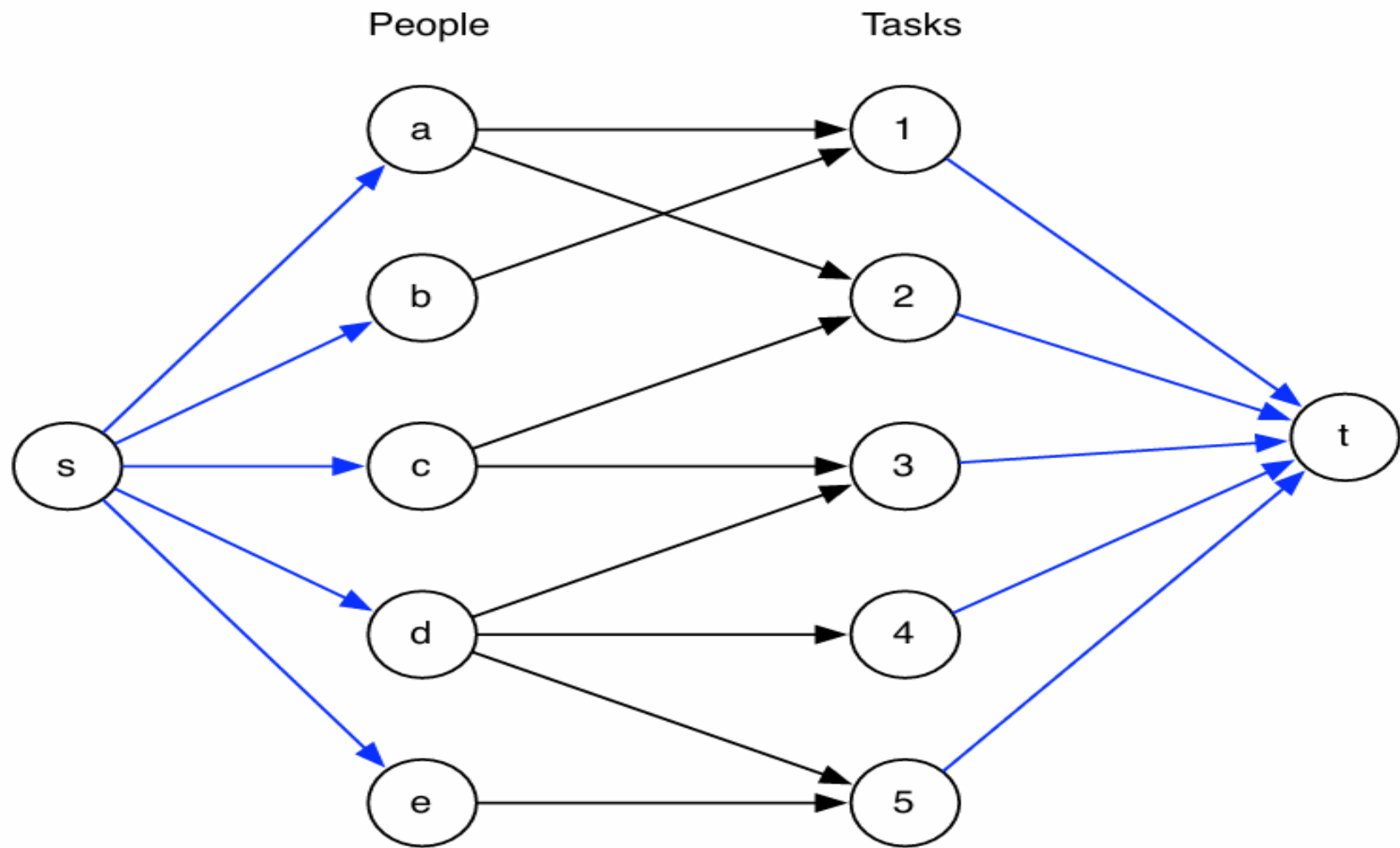


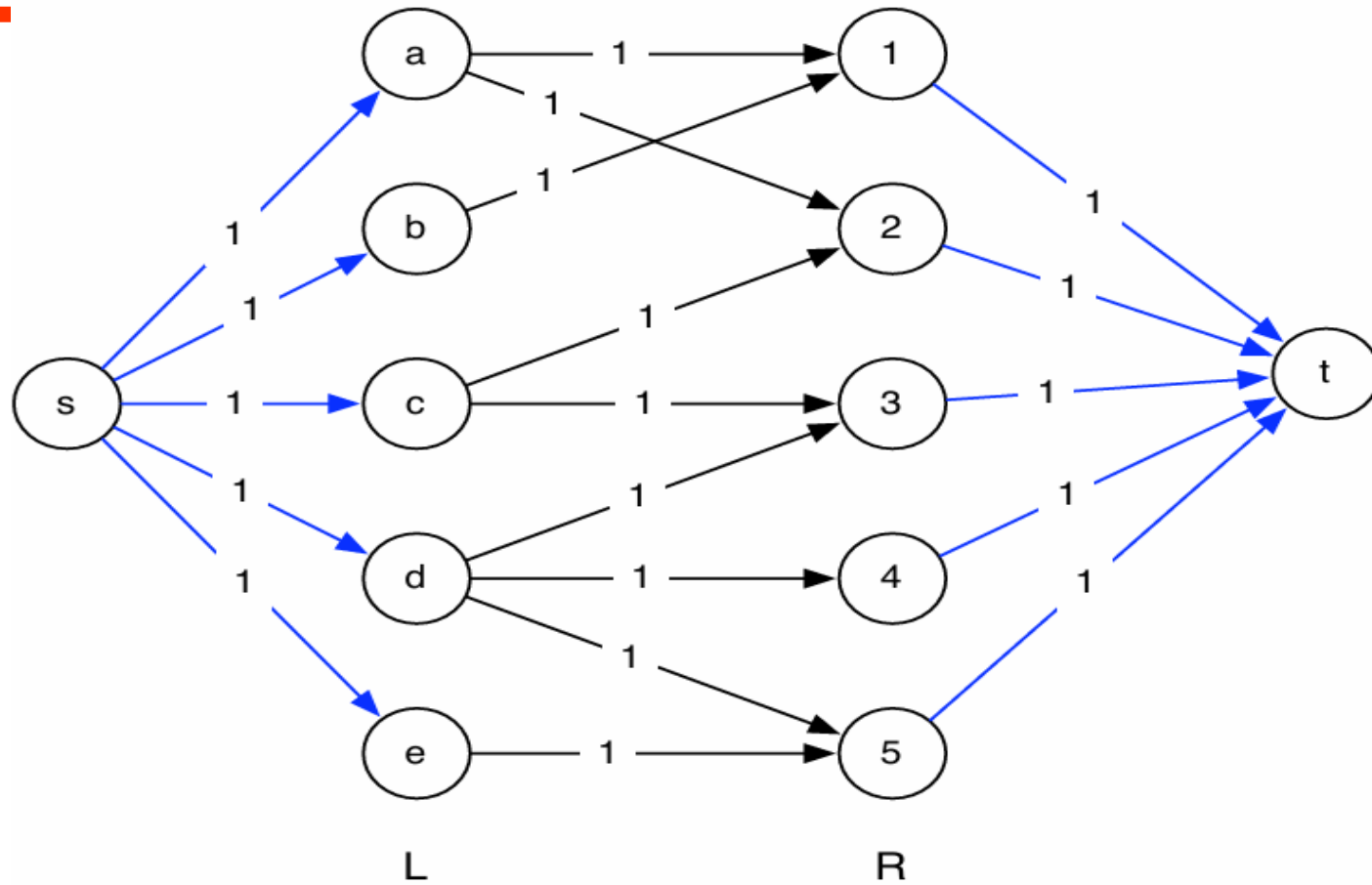
- 1 Given bipartite graph $G = (A \cup B, E)$, direct the edges from A to B .
- 2 Add new vertices s and t .
- 3 Add an edge from s to every vertex in A .
- 4 Add an edge from every vertex in B to t .
- 5 Make all the capacities 1.
- 6 Solve maximum network flow problem on this new graph G' .

Convert undirected to Directed graph



Reducing Bipartite Matching to Net Flow





$$E' = \{(s, u) : u \in L\} \cup \{(u, v) : (u, v) \in E\} \cup \{(v, t) : v \in R\}$$

Ford-Fulkerson can find a maximum matching in a bipartite graph in $O(V \cdot E)$ time.